

That is why the cloud is not one delivery option among several for quantum computing. It is essentially the only one. Nobody is putting a dilution refrigerator in their office in Pune, and the handful of organisations buying on-premises systems are national labs and a few very large banks doing long-horizon research.

The architecture is a queue, not a socket

Here is the part that surprises people coming from ordinary cloud work.

You do not open a connection to a QPU and issue instructions interactively. You submit a job. The job goes into a queue. At some point, minutes or hours later depending on demand, your circuit is loaded onto the hardware, executed a few thousand times, and the aggregated measurement results come back.

The path looks roughly like this. You write a circuit in Python on your laptop. A transpiler rewrites it to fit the target device, mapping your abstract qubits onto physical ones and decomposing your gates into the specific gate set that machine supports. The rewritten circuit is serialised, usually into OpenQASM, and posted to the provider's API. It waits in a queue. Control electronics eventually translates it into microwave or laser pulses. The results, which are just counts of measured bit strings, travel back.

This has two consequences.

Latency is not a network problem. Your round trip is dominated by queue time, which no amount of colocation fixes. If somebody tells you they need a QPU in a specific region for latency reasons, they have misunderstood the model.

And because results are statistical, you run the same circuit repeatedly. A thousand shots, ten thousand, more. Each run is a sample from a probability distribution, and you need enough samples to see the shape of it. This is why quantum jobs are billed per shot or per task rather than per second.

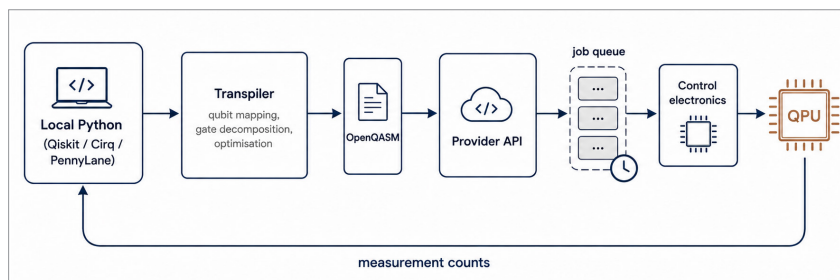


Figure 1: Measurement counts

Where the noise comes from

Skip this section if you already know it, but most explanations bury it and it matters for everything practical.

Qubits lose their quantum state. Fast. Coherence times on current hardware sit in the range of tens to hundreds of microseconds depending on the technology, and every gate operation you apply has an error rate attached, typically somewhere between one in a hundred and one in a thousand for two-qubit gates on good machines.

Errors compound. A circuit with a few hundred gates will accumulate enough errors that the output becomes indistinguishable from noise. That is the real limit on what you can run today, and it is why raw qubit counts are a poor way to compare machines. A five hundred qubit processor with mediocre gate fidelity may be less useful than a twenty qubit processor with excellent fidelity, because circuit depth is what you actually spend.

This is the NISQ (noisy intermediate-scale quantum) era, and the honest summary is that we are still in it. Error correction is progressing and the published roadmaps point at fault-tolerant machines around the end of this decade, but roadmaps are roadmaps. Plan for what exists.

Which brings me to the thing I say to every client who asks about this.

There is no verified case of a quantum computer beating a good classical algorithm on a real business problem today. Not one that has survived scrutiny. Several claimed results were later matched or beaten by

improved classical methods running on ordinary hardware. If a vendor presents you with a quantum advantage claim for your industry, ask what the best classical approach achieves on the same problem, and ask who checked.

This is not a reason to ignore the field. It is a reason to be precise about why you are engaging with it, which for almost everybody right now is capability building rather than production work.

Four ways of access

Access is consolidated into a small number of routes, and they split cleanly by business model.

IBM Quantum builds its own superconducting hardware and offers it directly. Open access to real devices, deep free tutorial material, and Qiskit as the native SDK. If you want a first account, this is where I send people, because the learning path is the most complete.

Google Quantum AI also builds its own hardware, around superconducting processors, but access is oriented towards research partners rather than general developers. Cirq is the SDK.

Amazon Braket is an aggregator. AWS builds little general-purpose QPU hardware itself and resells a marketplace of partner machines, currently spanning superconducting devices from IQM and Rigetti, trapped-ion systems from IonQ and AQT, and neutral-atom hardware from QuEra. If you already live in AWS, the IAM and S3 integration will feel familiar, and Braket Hybrid Jobs handles the classical-quantum loop.